

STA 5635 HW 7

Palmer Swanson

February 27, 2026

Problem 1

Batch size = 128.

(1a) NN w/ one hidden layer, 256 neurons, ReLU activation.

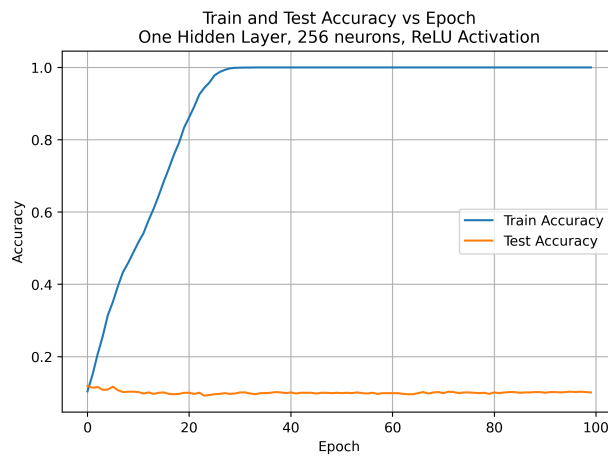


Figure 1: Train and Test accuracy vs. Epoch, part a.

(1b) NN w/ two hidden layers, 64 neurons each, ReLU activation

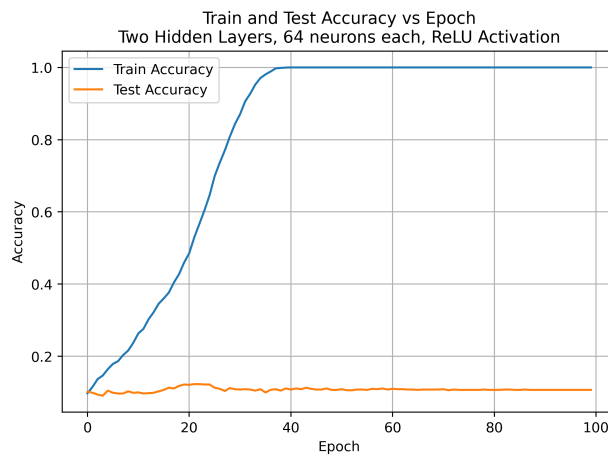


Figure 2: Train and Test accuracy vs. Epoch, part b.

(1c) CNN w/ one hidden layer, 32 filters of size 15, ReLU activation.

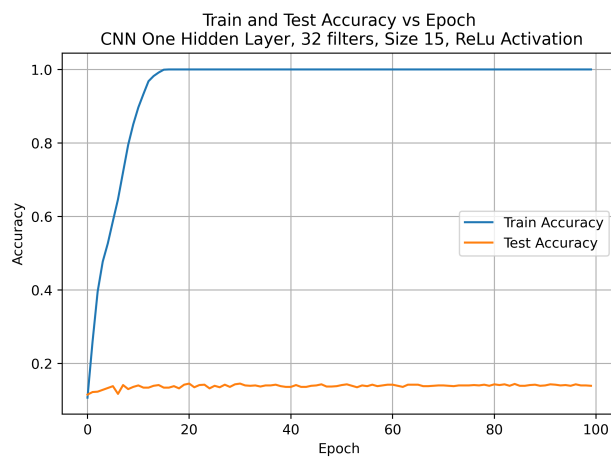


Figure 3: Train and Test accuracy vs. Epoch, part c.

(1d) CNN w/ one hidden layer, 32 filters of size 15, ReLU activation, max pooling size 2, stride 2

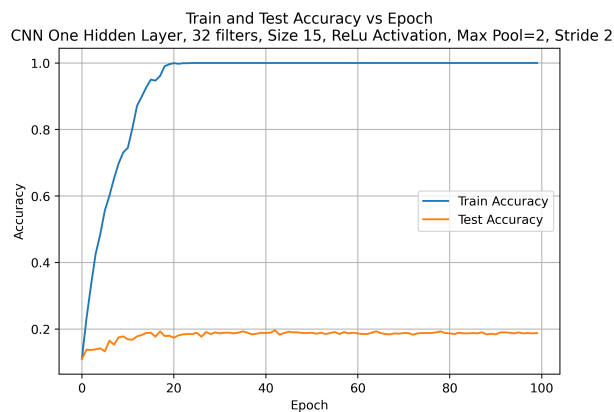


Figure 4: Train and Test accuracy vs. Epoch, part d.

- (1e) CNN w/ one hidden layer, 32 filters of size 15, ReLUE activation, max pooling entire output.

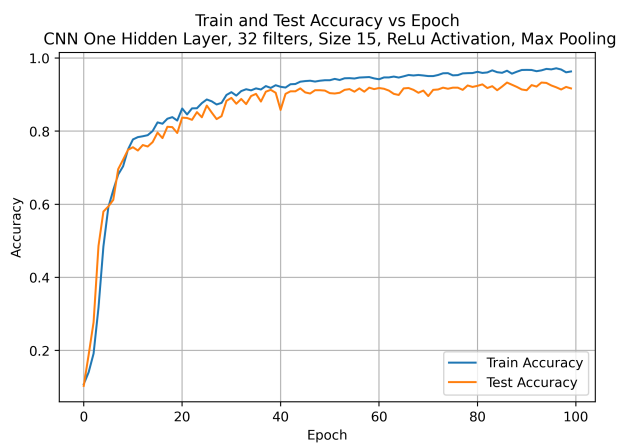


Figure 5: Train and Test accuracy vs. Epoch, part e.

- (1f) CNN w/ two hidden layers, 16 filters of size 15, ReLUE activation, max pooling size 2, stride 2.

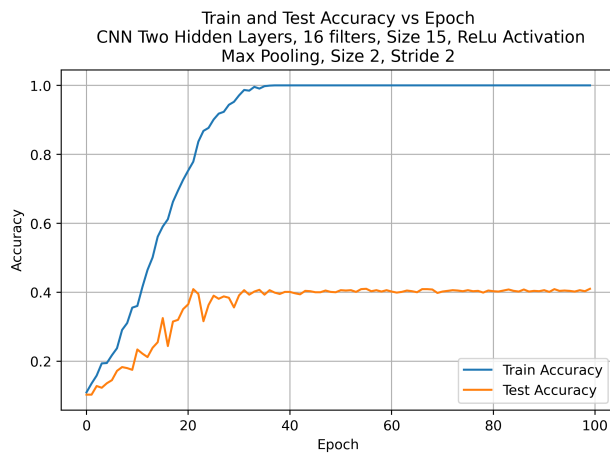


Figure 6: Train and Test accuracy vs. Epoch, part f.

- (1g) CNN w/ three hidden layers, 16 filters of size 15, ReLUE activation, max pooling size 2, stride 2.

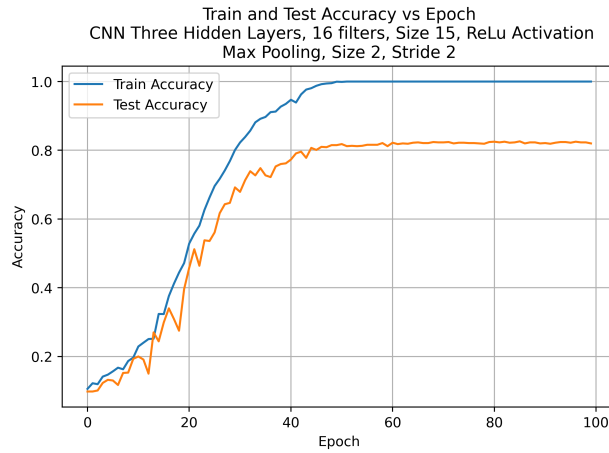


Figure 7: Train and Test accuracy vs. Epoch, part g.

- (1h) Table of final train and test accuracies. From this, it appears as though the model in

model	final_train_acc	final_test_acc
part_a	1.000000	0.101000
part_b	1.000000	0.107000
part_c	1.000000	0.139000
part_d	1.000000	0.188000
part_e	0.963333	0.917000
part_f	1.000000	0.410000
part_g	1.000000	0.820000

Table 1: Final Train and Test Accuracies for all Models

part e had the best test accuracy.

Code

```
1 import pandas as pd
2 import numpy as np
3 from scipy.io import loadmat
4 import torch
5 from torch import nn
6 from torch.utils.data import TensorDataset, DataLoader
7 import torch.optim as optim
8 import matplotlib.pyplot as plt
9
10 #https://pytorch.org/get-started/locally/
11 #https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html
12 #https://www.codecademy.com/article/building-a-neural-network-using-pytorch
13
14 #need to cut down the number of lines of code.
15 mat = loadmat("assignments/hw07/data/cnns1m3000.mat")
16
17 #one liners to get x and x test. stores them as tensors from the getgo
18 x, xtest = (torch.from_numpy(mat[k]) for k in ("x", "xtest"))
19 y, ytest = (torch.from_numpy(mat[k]).t().squeeze() for k in ("y", "ytest"))
20
21 #check the data types. mostly for debugging.
22 #print("x:", x.shape, x.dtype, "y:", y.shape, y.dtype)
23 #print("xtest:", xtest.shape, xtest.dtype, "ytest:", ytest.shape, ytest.dtype)
24
25 #Use GPU
26 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
27 print("Using device:", device)
28
29 #initialising the loss function and the number of epochs to use
30 #as well as optimizer to use. just picking one between 32 and 256
31 epochs_to_use = 100
32 batch_size = 128
33
34 #need to get Dataloaders of the Datasets
35 train_load = DataLoader(TensorDataset(x, y), batch_size=batch_size, shuffle=True)
36 test_load = DataLoader(TensorDataset(xtest, ytest), batch_size=batch_size,
37                        shuffle=False)
38
39 #function for reuse
40 #training the data.
41 def run_NN_train(model, loader, optimizer, criterion, device):
42     model.train()
43     correct, total = 0, 0
44
45     #batches
```

```

45     for x_batch, y_batch in loader:
46         x_batch = x_batch.to(device)
47         y_batch = y_batch.to(device)
48
49         #set gradients to 0, forward pass, compute the loss
50         #backward pass, then update model parameters
51         optimizer.zero_grad()
52         outputs = model(x_batch)
53         loss = criterion(outputs, y_batch)
54         loss.backward()
55         optimizer.step()
56
57         #make predictions
58         soft_outputs = torch.nn.functional.softmax(outputs, dim=1)
59         top_class = soft_outputs.topk(1, dim = 1)[1].squeeze()
60
61         correct = correct + (top_class == y_batch).sum().item()
62         total = total + y_batch.size(0)
63
64     return (correct/total)
65
66 #function for testing the data. Don't need optimizer anymore?
67 def run_NN_test(model, loader, device):
68     model.eval()
69     correct, total = 0,0
70
71     #batches
72     with torch.no_grad():
73         for x_batch, y_batch in loader:
74             x_batch = x_batch.to(device)
75             y_batch = y_batch.to(device)
76
77             #want predicted class labels
78             outputs = model(x_batch)
79             soft_outputs = torch.nn.functional.softmax(outputs, dim=1)
80             top_class = soft_outputs.topk(1, dim = 1)[1].squeeze()
81
82             correct = correct + (top_class == y_batch).sum().item()
83             total = total + y_batch.size(0)
84
85     return (correct/total)
86
87 #function to train and plot
88 def run_all(model, training_data, testing_data, device, epochs=100, lr=0.01,
89             momentum=0.9, print_option = 10, image_saved_path=None,
90             plot_title=""):
91     #send model to CUDA device, setting optimizer, and criterion

```

```

91     model = model.to(device)
92     optimizer = optim.SGD(model.parameters(), lr=lr, momentum=momentum)
93     criterion = nn.CrossEntropyLoss()
94
95     train_accs, test_accs = [], []
96     for epoch in range(epochs):
97         #for each epoch, train and test
98         train_acc = run_NN_train(model, train_load, optimizer,
criterion=criterion, device=device)
99         test_acc = run_NN_test(model, test_load, device)
100
101         train_accs.append(train_acc)
102         test_accs.append(test_acc)
103
104         if (epoch + 1) % print_option == 0:
105             print(f"Epoch_{epoch+1:3d}|_Train_acc:{train_acc:.4f}|_Test_acc:
{test_acc:.4f}")
106
107     #plotting stuff
108     plt.figure()
109     plt.plot(range(epochs), train_accs, label="Train_Accuracy")
110     plt.plot(range(epochs), test_accs, label="Test_Accuracy")
111     plt.xlabel("Epoch")
112     plt.ylabel("Accuracy")
113     plt.title(plot_title)
114     plt.legend()
115     plt.grid(True)
116     plt.tight_layout()
117     plt.savefig(image_saved_path, dpi=400, bbox_inches="tight")
118     plt.close()
119
120     return train_accs, test_accs
121
122 #information for a loop. List of Dicts
123 parts_a_b = [
124     dict(
125         name="part_a",
126         model=nn.Sequential(
127             nn.Linear(256, 256),
128             nn.ReLU(),
129             nn.Linear(256, 10)
130         ),
131         title="Train_and_Test_Accuracy_vs_Epoch\nOne_Hidden_Layer,_256_neurons,_
ReLU_Activation",
132         save_path="assignments/hw07/figures/part_a.png"
133     ),
134     dict(

```

```

135         name="part_b",
136         model=nn.Sequential(
137             nn.Linear(256, 64),
138             nn.ReLU(),
139             nn.Linear(64, 64),
140             nn.ReLU(),
141             nn.Linear(64, 10)
142         ),
143         title="Train and Test Accuracy vs Epoch\nTwo Hidden Layers, 64 neurons each, ReLU Activation",
144         save_path="assignments/hw07/figures/part_b.png"
145     )
146 ]
147
148
149 #parts a and b
150 results = []
151 for item in parts_a_b:
152     print(item["name"])
153     train_accuracy, test_accuracy = run_all(
154         model=item["model"],
155         training_data=train_load, testing_data=test_load,
156         device=device, plot_title=item["title"],
157         image_saved_path=item["save_path"], print_option=25
158     )
159
160     #add to the results object the final train and test accuracies
161     results.append({
162         "model": item["name"],
163         "final_train_acc": train_accuracy[-1],
164         "final_test_acc": test_accuracy[-1]
165     })
166
167
168 def run_CNN_train(model, loader, optimizer, criterion, device):
169     model.train()
170     correct, total = 0, 0
171
172     #batches
173     for x_batch, y_batch in loader:
174         x_batch = x_batch.to(device)
175         x_batch = x_batch.unsqueeze(1)
176         y_batch = y_batch.to(device)
177
178         #set gradients to 0, forward pass, compute the loss
179         #backward pass, then update model parameters

```



```

180     optimizer.zero_grad()
181     outputs = model(x_batch)
182     loss = criterion(outputs, y_batch)
183     loss.backward()
184     optimizer.step()
185
186     #make predictions
187     soft_outputs = torch.nn.functional.softmax(outputs, dim=1)
188     top_class = soft_outputs.topk(1, dim = 1)[1].squeeze()
189
190     correct = correct + (top_class == y_batch).sum().item()
191     total = total + y_batch.size(0)
192
193     return (correct/total)
194
195 #function for testing the data. Don't need optimizer anymore?
196 def run_CNN_test(model, loader, device):
197     model.eval()
198     correct, total = 0,0
199
200     #batches. Change here as opposed to the NN
201     with torch.no_grad():
202         for x_batch, y_batch in loader:
203             x_batch = x_batch.to(device)
204             x_batch = x_batch.unsqueeze(1)
205             y_batch = y_batch.to(device)
206
207             #want predicted class labels
208             outputs = model(x_batch)
209             soft_outputs = torch.nn.functional.softmax(outputs, dim=1)
210             top_class = soft_outputs.topk(1, dim = 1)[1].squeeze()
211
212             correct = correct + (top_class == y_batch).sum().item()
213             total = total + y_batch.size(0)
214
215         return (correct/total)
216
217 #function to train and plot
218 def run_all_CNN(model, training_data, testing_data, device, epochs=100, lr=0.01,
219                momentum=0.9, print_option = 10, image_saved_path=None,
220                plot_title=""):
221     #send model to CUDA device, setting optimizer, and criterion
222     model = model.to(device)
223     optimizer = optim.SGD(model.parameters(), lr=lr, momentum=momentum)
224     criterion = nn.CrossEntropyLoss()
225
226     train_accs, test_accs = [], []

```

```

226     for epoch in range(epochs):
227         #for each epoch, train and test
228         train_acc = run_CNN_train(model, train_load, optimizer,
criterion=criterion, device=device)
229         test_acc = run_CNN_test(model, test_load, device)
230
231         train_accs.append(train_acc)
232         test_accs.append(test_acc)
233
234         if (epoch + 1) % print_option == 0:
235             print(f"Epoch_{epoch+1:3d}|_Train_acc:{train_acc:.4f}|_Test_acc:
{test_acc:.4f}")
236
237     #plotting stuff
238     plt.figure()
239     plt.plot(range(epochs_to_use), train_accs, label="Train_Accuracy")
240     plt.plot(range(epochs_to_use), test_accs, label="Test_Accuracy")
241     plt.xlabel("Epoch")
242     plt.ylabel("Accuracy")
243     plt.title(plot_title)
244     plt.legend()
245     plt.grid(True)
246     plt.tight_layout()
247     plt.savefig(image_saved_path, dpi=400, bbox_inches="tight")
248     plt.close()
249
250     return train_accs, test_accs
251
252 #list of dicts for the CNN. needed to unsqueeze data for the Conv1d filter
253 parts_c_g = [
254     dict(
255         name="part_c",
256         model = nn.Sequential(
257             nn.Conv1d(in_channels=1, out_channels=32, kernel_size=15),
258             nn.ReLU(),
259             nn.Flatten(),
260             nn.Linear(32*242, 10)
261         ),
262         title="Train_and_Test_Accuracy_vs_Epoch\nCNN_One_Hidden_Layer,_32_
filters,_Size_15,_ReLu_Activation",
263         save_path="assignments/hw07/figures/part_c.png"
264     ),
265     dict(
266         name="part_d",
267         model = nn.Sequential(
268             nn.Conv1d(in_channels=1, out_channels=32, kernel_size=15),
269             nn.ReLU(),

```

```

270 #https://docs.pytorch.org/docs/stable/generated/torch.nn.MaxPool1d.html
271     nn.MaxPool1d(kernel_size=2, stride=2),
272     nn.Flatten(),
273     nn.LazyLinear(10)
274 ),
275     title="Train_and_Test_Accuracy_vs_Epoch\nCNN_One_Hidden_Layer,32_
filters,_Size_15,_ReLu_Activation,_Max_Pool=2,_Stride_2",
276     save_path="assignments/hw07/figures/part_d.png"
277 ),
278 dict(
279     name="part_e",
280     model = nn.Sequential(
281         nn.Conv1d(in_channels=1, out_channels=32, kernel_size=15),
282         nn.ReLU(),
283         #input size 256. Kernel size 15. 256-15=242.
284         #across entire size of map
285
286         #https://medium.com/@minhazc.engg/padding-and-strides-in-cnn-58dc56493887.
287         Stride = kernel size = global pooling
288         nn.MaxPool1d(kernel_size=242, stride=242),
289         nn.Flatten(),
290         nn.Linear(32, 10)
291     ),
292     title="Train_and_Test_Accuracy_vs_Epoch\nCNN_One_Hidden_Layer,32_
filters,_Size_15,_ReLu_Activation,_Max_Pooling",
293     save_path="assignments/hw07/figures/part_e.png"
294 ),
295 dict(
296     name="part_f",
297     model = nn.Sequential(
298         #layer 1
299         nn.Conv1d(in_channels=1, out_channels=16, kernel_size=15),
300         nn.ReLU(),
301         nn.MaxPool1d(kernel_size=2, stride=2),
302
303         #layer 2
304         nn.Conv1d(in_channels=16, out_channels=16, kernel_size=15),
305         nn.ReLU(),
306         nn.MaxPool1d(kernel_size=2, stride=2),
307
308         nn.Flatten(),
309         nn.LazyLinear(10)
310     ),
311     title="Train_and_Test_Accuracy_vs_Epoch\nCNN_Two_Hidden_Layers,_16_
filters,_Size_15,_ReLu_Activation\nMax_Pooling,_Size_2,_Stride_2",
312     save_path="assignments/hw07/figures/part_f.png"

```

```

311     ),
312     dict(
313         name="part_g",
314         model = nn.Sequential(
315             #layer 1
316             nn.Conv1d(in_channels=1, out_channels=16, kernel_size=15),
317             nn.ReLU(),
318             nn.MaxPool1d(kernel_size=2, stride=2),
319
320             #layer 2
321             nn.Conv1d(in_channels=16, out_channels=16, kernel_size=15),
322             nn.ReLU(),
323             nn.MaxPool1d(kernel_size=2, stride=2),
324
325             #layer 3
326             nn.Conv1d(in_channels=16, out_channels=16, kernel_size=15),
327             nn.ReLU(),
328             nn.MaxPool1d(kernel_size=2, stride=2),
329
330             nn.Flatten(),
331             nn.Linear(10)
332         ),
333         title="Train_and_Test_Accuracy_vs_Epoch\nCNN_Three_Hidden_Layers,_16_filters,_Size_15,_ReLU_Activation\nMax_Pooling,_Size_2,_Stride_2",
334         save_path="assignments/hw07/figures/part_g.png"
335     )
336 ]
337
338 #parts c through g
339 results_cnn = []
340 for item in parts_c_g:
341     print(item["name"])
342     train_accuracy, test_accuracy = run_all_CNN(
343         model=item["model"],
344         training_data=train_load, testing_data=test_load,
345         device=device, plot_title=item["title"],
346         image_saved_path=item["save_path"], print_option=25
347     )
348
349     #add to the results object the final train and test accuracies
350     results_cnn.append({
351         "model": item["name"],
352         "final_train_acc": train_accuracy[-1],
353         "final_test_acc": test_accuracy[-1]
354     })
355
356 #part h. One big table

```

```

356 total_results = pd.DataFrame(results + results_cnn)
357 print(total_results)
358
359 #need to send this table to LATEX
360 LATEX_table = total_results.to_latex(
361     index=False,
362     caption=None,
363     label=None,
364     column_format="c" * total_results.shape[1],
365     escape=True
366 )
367
368 #save latex table
369 with open("assignments/hw07/output/final_train_test_acc.tex", "w") as f:
370     f.write(LATEX_table)
371
372 #probably could've fit this all into one big loop, but oh well.

```